Som Javia

Assignment 4 Code Analysis:

- A. xAI's Grok and Claude
- B. Went to their websites and typed out my prompt
- C. Prompt: You will be creating a sudoku puzzle solver written in C++ using objects NOT FUNCTIONS, to implement the requirements, make sure to use .txt files to complete this task, you have 5 puzzle txt files, 1 solution txt and some of the screenshots that should give you context on what to do.
  Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints: each digit from 1-9 must appear exactly once in each row, column, and block. Published puzzles are usually constructed to have a unique solution that requires at least 17 cells to be filled in (may require more depending on the puzzle and which cells are empty as many 78 filled cells can be required to guarantee uniqueness). Given a partially completed sudoku puzzle, find the (or at least a) solution, if no solution exists please prove it.
  We're going to implement a brute force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. For example, Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and returned a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier)
  This approach is depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
  You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
  Write a recursive object-oriented program to find the solution to the five puzzle files, with the output for each solution including puzzle file name, puzzle from file printed out, and solution to puzzle printed out if no solution is found "No solution found" printed out. Solution may not be unique, if there's more than 1

solution to the puzzle the program should print them out.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- D. Generated Code:
  - Grok:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cctype>

// Cell represents a single square on the board
class Cell {
private:
    int value;          // 0 means empty
    bool fixed;         // true if the value came from the
puzzle file

public:
    Cell() : value(0), fixed(false) {}
    Cell(int v, bool f) : value(v), fixed(f) {}
```

```
    int getValue() const { return value; }
    void setValue(int v) { value = v; }
    bool isFixed() const { return fixed; }
    void setFixed(bool f) { fixed = f; }
    bool isEmpty() const { return value == 0; }
};

// Board owns the 9x9 grid of Cells
class Board {
private:
    Cell grid[9][9];

public:
    Board() {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                grid[r][c] = Cell();
    }

    // Copy constructor - needed when we store complete
solutions
    Board(const Board& other) {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                grid[r][c] = other.grid[r][c];
    }

    Board& operator=(const Board& other) {
        if (this != &other) {
            for (int r = 0; r < 9; ++r)
                for (int c = 0; c < 9; ++c)
                    grid[r][c] = other.grid[r][c];
        }
```

```
        return *this;
    }

    Cell& at(int row, int col) { return grid[row][col]; }
    const Cell& at(int row, int col) const { return
grid[row][col]; }

    // Load a puzzle from a text file. Underscores (or any
non-digit) become empty cells.
    bool loadFromFile(const std::string& filename) {
        std::ifstream in(filename.c_str());
        if (!in) {
            std::cerr << "Cannot open file: " << filename <<
std::endl;
            return false;
        }

        int row = 0, col = 0;
        char ch;
        while (in.get(ch) && row < 9) {
            if (ch == '_' || ch == '.') {
                grid[row][col] = Cell(0, false);
                ++col;
            } else if (std::isdigit(static_cast<unsigned
char>(ch))) {
                int v = ch - '0';
                if (v >= 1 && v <= 9) {
                    grid[row][col] = Cell(v, true);
                    ++col;
                }
            }
            // ignore whitespace and any other characters

            if (col == 9) {
```

```
                col = 0;
                ++row;
            }
        }
        in.close();
        return row == 9;   // successfully read 9 rows
    }

    void print(std::ostream& out = std::cout) const {
        for (int r = 0; r < 9; ++r) {
            if (r == 3 || r == 6) out <<
"------+-------+------\n";
            for (int c = 0; c < 9; ++c) {
                if (c == 3 || c == 6) out << "| ";
                int v = grid[r][c].getValue();
                if (v == 0) out << "_ ";
                else        out << v << ' ';
            }
            out << '\n';
        }
    }

    // Check whether placing num at (row,col) violates any
constraint
    bool isSafe(int row, int col, int num) const {
        // row & column
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i].getValue() == num) return false;
            if (grid[i][col].getValue() == num) return false;
        }
        // 3x3 block
        int br = (row / 3) * 3;
        int bc = (col / 3) * 3;
```

```
        for (int r = br; r < br + 3; ++r)
            for (int c = bc; c < bc + 3; ++c)
                if (grid[r][c].getValue() == num) return
false;
        return true;
    }

    // Locate the next empty cell; returns false if the board
is full
    bool findEmpty(int& row, int& col) const {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                if (grid[r][c].isEmpty()) {
                    row = r;
                    col = c;
                    return true;
                }
        return false;
    }
};

// Solver performs the depth-first search with backtracking
class Solver {
private:
    Board board;
    std::vector<Board> solutions;

    // Recursive backtracking routine. Continues after finding
a solution
    // so that every possible solution is collected.
    void search() {
        int row, col;
        if (!board.findEmpty(row, col)) {
```

```
            // board is complete - store a copy
            solutions.push_back(board);
            return;
        }

        for (int num = 1; num <= 9; ++num) {
            if (board.isSafe(row, col, num)) {
                board.at(row, col).setValue(num);
                search();
                board.at(row, col).setValue(0);   // backtrack
            }
        }
    }

public:
    explicit Solver(const Board& b) : board(b) {}

    void solveAll() {
        solutions.clear();
        search();
    }

    const std::vector<Board>& getSolutions() const { return
solutions; }

    void printResults(const std::string& filename, const
Board& original) const {
        std::cout <<
"======================================\n";
        std::cout << "Puzzle file: " << filename << "\n";
        std::cout <<
"--------------------------------------\n";
        std::cout << "Original puzzle:\n";
        original.print();
```

```
        std::cout <<
"----------------------------------------\n";

        if (solutions.empty()) {
            std::cout << "No solution found\n";
        } else {
            std::cout << "Solutions found: " <<
solutions.size() << "\n\n";
            for (size_t i = 0; i < solutions.size(); ++i) {
                std::cout << "Solution #" << (i + 1) << ":\n";
                solutions[i].print();
                if (i + 1 < solutions.size()) std::cout <<
'\n';
            }
        }
        std::cout <<
"========================================\n\n";
    }
};

// Application object - owns the list of puzzle files and
drives the process
class SudokuApplication {
private:
    std::vector<std::string> puzzleFiles;

public:
    SudokuApplication() {

puzzleFiles.push_back("/home/workdir/attachments/puzzle1.txt")
;

puzzleFiles.push_back("/home/workdir/attachments/puzzle2.txt")
;
```

```
puzzleFiles.push_back("/home/workdir/attachments/puzzle3.txt")
;

puzzleFiles.push_back("/home/workdir/attachments/puzzle4.txt")
;

puzzleFiles.push_back("/home/workdir/attachments/puzzle5.txt")
;
    }

    void run() {
        for (size_t i = 0; i < puzzleFiles.size(); ++i) {
            const std::string& file = puzzleFiles[i];
            Board board;
            if (!board.loadFromFile(file)) {
                std::cerr << "Failed to load " << file <<
std::endl;
                continue;
            }

            // Keep a pristine copy for printing the original
puzzle
            Board original = board;

            Solver solver(board);
            solver.solveAll();
            solver.printResults(file, original);
        }
    }
};

int main() {
    SudokuApplication app;
```

```
    app.run();
    return 0;
}
```

- Claude: `// Sudoku solver: brute-force depth-first search with backtracking.`

```
// Object-oriented design: Board holds the grid, Solver does the recursive search.
//
// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp
// Run:    ./sudoku puzzle1.txt puzzle2.txt ...   (defaults to puzzle1..5.txt)

#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

//
// ----------------------------------------------------------------------------
// Board: a 9x9 grid. 0 means an empty cell.
//
// ----------------------------------------------------------------------------
class Board {
public:
    static const int N = 9;

    Board() { cells_.fill(0); }
```

```
    // Reads 81 tokens: digits 1-9 or '_' for blank. Returns
false on bad input.
    bool load(const std::string& filename) {
        std::ifstream in(filename);
        if (!in) return false;
        std::string tok;
        int count = 0;
        while (count < N * N && in >> tok) {
            if (tok == "_") cells_[count++] = 0;
            else if (tok.size() == 1 && tok[0] >= '1' &&
tok[0] <= '9')
                cells_[count++] = tok[0] - '0';
            else return false;
        }
        return count == N * N;
    }

    int get(int r, int c) const { return cells_[r * N + c]; }
    void set(int r, int c, int v) { cells_[r * N + c] = v; }

    // Can digit v be placed at (r,c) without breaking
row/column/block rules?
    bool canPlace(int r, int c, int v) const {
        for (int i = 0; i < N; ++i) {
            if (get(r, i) == v) return false;
            if (get(i, c) == v) return false;
        }
        int br = (r / 3) * 3, bc = (c / 3) * 3;
        for (int i = 0; i < 3; ++i)
            for (int j = 0; j < 3; ++j)
                if (get(br + i, bc + j) == v) return false;
        return true;
    }
```

```
    // Are the given clues consistent (no duplicates in any
row/col/block)?
    bool cluesConsistent() {
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c) {
                int v = get(r, c);
                if (v == 0) continue;
                set(r, c, 0);
                bool ok = canPlace(r, c, v);
                set(r, c, v);
                if (!ok) return false;
            }
        return true;
    }

    // Finds the first empty cell; returns false if the board
is full.
    bool findEmpty(int& r, int& c) const {
        for (r = 0; r < N; ++r)
            for (c = 0; c < N; ++c)
                if (get(r, c) == 0) return true;
        return false;
    }

    void print(std::ostream& os) const {
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                int v = get(r, c);
                os << (v ? char('0' + v) : '_') << ' ';
            }
            os << '\n';
        }
```

```
    }

private:
    std::array<int, N * N> cells_;
};

//
------------------------------------------------------------------
-------------
// Solver: recursive backtracking that collects ALL solutions.
//
------------------------------------------------------------------
-------------
class Solver {
public:
    explicit Solver(const Board& puzzle) : board_(puzzle) {}

    // Returns every solution found (empty vector => no
solution).
    const std::vector<Board>& solveAll() {
        solutions_.clear();
        if (board_.cluesConsistent()) search();
        return solutions_;
    }

private:
    void search() {
        int r, c;
        if (!board_.findEmpty(r, c)) {      // k == 81:
complete solution
            solutions_.push_back(board_);
            return;
        }
```

```
        // Try each candidate; no candidates => dead end,
return (backtrack).
        for (int v = 1; v <= 9; ++v) {
            if (board_.canPlace(r, c, v)) {
                board_.set(r, c, v);
                search();                   // recursive call
with k+1 filled
                board_.set(r, c, 0);        // undo and try
next candidate
            }
        }
    }

    Board board_;
    std::vector<Board> solutions_;
};

//
----------------------------------------------------------------
-------------
// PuzzleRunner: handles one file end to end (load, solve,
report).
//
----------------------------------------------------------------
-------------
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string& filename) :
filename_(filename) {}

    void run() {
        std::cout << "Puzzle file: " << filename_ << "\n";
        Board puzzle;
        if (!puzzle.load(filename_)) {
```

```
            std::cout << "Could not read a valid puzzle from
this file.\n\n";
            return;
        }
        std::cout << "Puzzle:\n";
        puzzle.print(std::cout);

        Solver solver(puzzle);
        const std::vector<Board>& sols = solver.solveAll();

        if (sols.empty()) {
            std::cout << "\nNo solution found\n";
        } else {
            for (size_t i = 0; i < sols.size(); ++i) {
                std::cout << "\nSolution " << (i + 1) <<
":\n";
                sols[i].print(std::cout);
            }
            std::cout << "\nSolutions found: " << sols.size()
<< "\n";
        }
        std::cout <<
"\n----------------------------------------\n\n";
    }

private:
    std::string filename_;
};

int main(int argc, char* argv[]) {
    std::vector<std::string> files;
    for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    if (files.empty())
        for (int i = 1; i <= 5; ++i)
```

```
            files.push_back("puzzle" + std::to_string(i) +
".txt");


    for (const std::string& f : files) PuzzleRunner(f).run();
    return 0;
}
```

- E. In terms of time complexity, both codes report the worst time case would be O(9^k), with k leading to empty cells that are up to 81, most of the puzzles are finished in under a second. Most puzzles are able to be solved in this time, with harder puzzles being recommended to be optimized. Space complexity is mostly O(k) + O(s) with k being a part of the recursion within the 81 cells and O(s) being for s solutions which are unbounded. For maintainability, code is even properly with different responsibilities for board, the cells, and solving the sudoku puzzle.
- F. I used Claude code because of its simplistic nature of organizing responsibilities into a concise manner in the code and also being able to summarize large blocks of code from its comments.
- G. Even though I admire Claude's code a lot, I liked the output of Groks code more, so I decided to change Claude's output code to match Groks output code and there I was satisfied. Made the output a lot cleaner, and also fixed some unnecessary one line if else statements in some of Claudes code.